

Học viện Công nghệ Bưu chính Viễn thông — Khoa Công nghệ Thông tin

Phát triển Hệ thống Thông minh (Intelligent Systems Development)

BÁO CÁO

## Diabetes 130-US hospitals

Mạng MLP dự đoán tái nhập viện, cài đặt ba cách — NumPy thuần,  
TensorFlow/Keras và PyTorch

Bài tập 4 — So sánh ba cách cài đặt CNN

|                     |                       |
|---------------------|-----------------------|
| <b>Sinh viên</b>    | Trần Xuân Thành       |
| <b>Mã sinh viên</b> | B23DCAT280            |
| <b>Lớp</b>          | E23CNPM01 — Nhóm 02   |
| <b>Giảng viên</b>   | PGS.TS. Trần Đình Quế |

Tháng 9 năm 2026

# Mục lục

---

|                                                    |           |
|----------------------------------------------------|-----------|
| Vì sao ở đây dùng MLP chứ không dùng CNN . . . . . | 2         |
| Những gì giữ nguyên ở cả ba cách cài đặt . . . . . | 2         |
| <b>1. Dữ liệu</b>                                  | <b>3</b>  |
| <b>2. Kiến trúc dùng chung</b>                     | <b>4</b>  |
| Đếm tham số bằng tay trước đã . . . . .            | 4         |
| <b>3. Cách cài đặt A — viết từ đầu (NumPy)</b>     | <b>5</b>  |
| Đạo hàm viết tay có đúng thật không? . . . . .     | 6         |
| <b>4. Cách cài đặt B — TensorFlow / Keras</b>      | <b>8</b>  |
| <b>5. Cách cài đặt C — PyTorch</b>                 | <b>10</b> |
| <b>6. So sánh</b>                                  | <b>12</b> |
| 6.1 Những con số slide 29 yêu cầu . . . . .        | 12        |
| 6.2 Mỗi thành phần nằm ở đâu . . . . .             | 14        |
| <b>Lưu lại mô hình đã huấn luyện</b>               | <b>16</b> |
| <b>7. Bộ dữ liệu này cho thấy điều gì</b>          | <b>17</b> |

## Một bài toán học máy, ba cách cài đặt: NumPy thuần, TensorFlow/Keras và PyTorch.

Notebook này làm bộ dữ liệu đầu tiên trong ba bộ mà đề bài yêu cầu:

| # | Bộ dữ liệu                       | Cấu trúc                     | Họ mô hình |
|---|----------------------------------|------------------------------|------------|
| 1 | <b>Diabetes 130-US hospitals</b> | bảng, 179 đặc trưng          | <b>MLP</b> |
| 2 | MNIST                            | ảnh, $1 \times 28 \times 28$ | CNN        |
| 3 | CIFAR-10                         | ảnh, $3 \times 32 \times 32$ | CNN        |

### Vì sao ở đây dùng MLP chứ không dùng CNN

Slide 4 của bài giảng 04 nêu đúng cái phân biệt làm nền cho cả bài tập này. Một dòng dữ liệu bảng

$$x = [x_1, x_2, \dots, x_d]$$

không có khái niệm lân cận hai chiều: đặc trưng  $x_i$  không nằm cạnh  $x_{i+1}$  theo nghĩa không gian như hai điểm ảnh kề nhau. Phép tích chập (convolution) sinh ra để khai thác **cấu trúc không gian cục bộ**; chỗ nào không có cấu trúc ấy thì hai ý tưởng cốt lõi của nó — kết nối cục bộ và chia sẻ trọng số — chẳng mua được gì. Vậy nên:

**Diabetes** → MLP

**MNIST / CIFAR-10** → CNN

**Mô hình phải khớp với cấu trúc của dữ liệu.** Câu đó chính là lý do bộ dữ liệu này có mặt trong bài tập: nó là ca đối chứng cho thấy CNN là một lựa chọn có lý do, không phải thứ mặc định.

### Những gì giữ nguyên ở cả ba cách cài đặt

Slide 29 đặt ra quy tắc công bằng:

*Cùng dữ liệu + cùng phép chia + cùng kiến trúc + siêu tham số tương đương*

Cả ba nhánh dưới đây đều lấy mảng dữ liệu từ cùng một hàm nạp trong `ass4_utils` với cùng một hạt ngẫu nhiên (seed), nên tập huấn luyện và tập kiểm tra giống nhau đến từng byte. Thứ duy nhất thay đổi là **mức trừu tượng**.

```
import os, sys, json, time, platform

# TensorFlow is CPU-only on Windows (no GPU support since 2.10); silence its startup
↪ chatter.
os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

sys.path.insert(0, os.path.abspath("."))
import ass4_utils as U
import scratch_nn as S

U.set_seed(U.SEED)

import torch
import tensorflow as tf
import keras

print("python      ", platform.python_version())
print("numpy        ", np.__version__)
```

```
print("torch      ", torch.__version__, "| CUDA:", torch.cuda.is_available(),
      "|", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU only")
print("tensorflow ", tf.__version__, "| GPUs:", tf.config.list_physical_devices("GPU") or
      ↪ "none (CPU)")
print("keras      ", keras.__version__)

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
plt.rcParams["figure.dpi"] = 110
```

```
python      3.11.8
numpy       2.2.6
torch       2.10.0+cu130 | CUDA: True | NVIDIA GeForce RTX 3060 Laptop GPU
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11.
↪ Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the
↪ TensorFlow-DirectML plugin.

tensorflow  2.21.0 | GPUs: none (CPU)
keras       3.15.1
```

## 1. Dữ liệu

Bộ dữ liệu là bản *Diabetes 130-US hospitals for years 1999–2008* của UCI: mười năm hồ sơ bệnh nhân nội trú, **101.766 lượt khám** × **50 cột**. Dữ liệu tự tải về ở lần chạy đầu tiên.

Hàm nạp trong `ass4_utils.py` áp dụng các bước tiền xử lý chuẩn của bộ dữ liệu này:

1. **Mỗi bệnh nhân một dòng.** Các lượt khám lặp lại của cùng một người làm rò rỉ kết quả của lần nhập viện trước sang lần sau, nên chỉ giữ lần nhập viện đầu tiên.
2. **Bỏ các ca tử vong / chuyển chăm sóc cuối đời.** Những bệnh nhân này không thể tái nhập viện; gán nhãn NO cho họ là dạy mô hình một quy luật sai.
3. **Bỏ cột định danh và cột gần như rỗng** — `weight` (thiếu 96,9 %), `payer_code` (39,6 %), `medical_specialty` (49,1 %), cùng hai cột thuốc hằng số `examide` và `citoglipton`.
4. **Gom mã chẩn đoán ICD-9.** Ba cột `diag_1..3` mỗi cột chứa khoảng 700 mã khác nhau; chúng được gom về chín nhóm lâm sàng (tuần hoàn, hô hấp, tiểu đường, ...) để phép mã hoá one-hot không bùng nổ chiều.
5. **Mã hoá one-hot biến hạng mục, chuẩn hoá biến số,** chỉ dùng thống kê của **tập huấn luyện**.

Biến mục tiêu là `readmitted` với ba lớp: NO, >30 ngày, <30 ngày.

```
X_train, y_train, X_test, y_test, meta = U.load_diabetes130()
```

```
N_FEATURES = X_train.shape[1]
CLASSES = meta["classes"]
N_CLASSES = len(CLASSES)
```

```
print(f"\ntrain {X_train.shape}   test {X_test.shape}   dtype {X_train.dtype}")
print(f"features: {N_FEATURES}   classes: {CLASSES}")
```

```
Diabetes 130-US hospitals: raw (101766, 50) -> (69973, 179), train 55979 / test 13994, 179 features,
↪ 3 classes
```

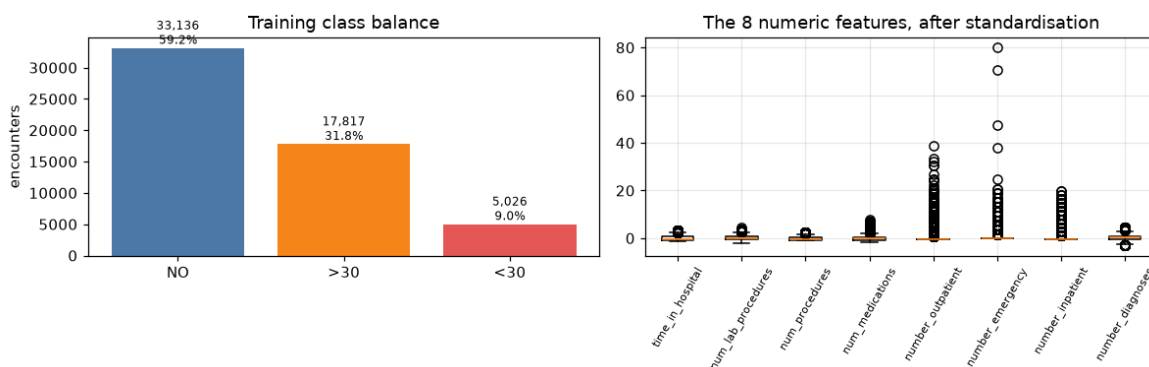
```
train (55979, 179)   test (13994, 179)   dtype float32
features: 179   classes: ['NO', '>30', '<30']
```

```
# Class balance and a look at the feature block structure.
fig, ax = plt.subplots(1, 2, figsize=(11, 3.6))
```

```
counts = np.bincount(y_train, minlength=N_CLASSES)
bars = ax[0].bar(CLASSES, counts, color=["#4C78A8", "#F58518", "#E45756"])
ax[0].set_title("Training class balance")
ax[0].set_ylabel("encounters")
for b, c in zip(bars, counts):
    ax[0].text(b.get_x() + b.get_width() / 2, c, f"{c:,}\n{c/counts.sum():.1%}",
               ha="center", va="bottom", fontsize=8)

numeric_names = [n for n in meta["feature_names"][:8]]
ax[1].boxplot([X_train[:, i] for i in range(8)], tick_labels=numeric_names)
ax[1].set_title("The 8 numeric features, after standardisation")
ax[1].tick_params(axis="x", rotation=60, labels=7)
ax[1].grid(alpha=0.3)
fig.tight_layout()
plt.show()

print(f"{'class':<6}{'train':>10}{'share':>9}")
for i, c in enumerate(CLASSES):
    print(f"{'c':<6}{counts[i]:>10},{counts[i]/counts.sum():>9.1%}")
```



| class | train  | share |
|-------|--------|-------|
| NO    | 33,136 | 59.2% |
| >30   | 17,817 | 31.8% |
| <30   | 5,026  | 9.0%  |

*Mất cân bằng lớp ảnh hưởng tới cách đọc kết quả ở phần sau. Chỉ cần đoán NO cho mọi bệnh nhân là đã đạt khoảng 59 % độ chính xác, trong khi về mặt lâm sàng thì vô dụng. Đó là lý do các bảng so sánh luôn báo **macro F1** bên cạnh độ chính xác: macro F1 lấy trung bình đều trên cả ba lớp, nên mô hình bỏ qua lớp hiếm <30 không thể núp sau lớp đa số.*

## 2. Kiến trúc dùng chung

Mọi cách cài đặt bên dưới đều dựng đúng **hàm này**, không thêm gì khác:

$X \rightarrow \text{Linear}_{179 \rightarrow 128} \rightarrow \text{ReLU} \rightarrow \text{Dropout}_{0.3} \rightarrow \text{Linear}_{128 \rightarrow 64} \rightarrow \text{ReLU} \rightarrow \text{Linear}_{64 \rightarrow 3} \rightarrow \hat{y}$

Đây chính là luận điểm trung tâm của tài liệu lý thuyết, viết ra cụ thể — một mạng nơ-ron là một hợp thành của các hàm số:

$$F = f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1$$

### Đếm tham số bằng tay trước đã

Mục §43 của tài liệu cho công thức của một tầng tuyến tính:

$$N_{\text{params}} = N_{\text{in}}N_{\text{out}} + N_{\text{out}}$$

Làm phép tính này **trước khi** hỏi bất kỳ framework nào mới là điểm mấu chốt của bài: sau đó từng cách cài đặt được đối chiếu với con số này, và cả ba phải khớp.

```
HIDDEN1, HIDDEN2, DROPOUT = 128, 64, 0.3
EPOCHS, BATCH_SIZE, LR = 15, 256, 1e-3

# Hand-computed, per tutorial section 43: N_in * N_out + N_out
manual = [
    ("Linear 179->128", N_FEATURES * HIDDEN1 + HIDDEN1),
    ("ReLU", 0),
    ("Dropout 0.3", 0),
    ("Linear 128->64", HIDDEN1 * HIDDEN2 + HIDDEN2),
    ("ReLU", 0),
    ("Linear 64->3", HIDDEN2 * N_CLASSES + N_CLASSES),
]
EXPECTED_PARAMS = sum(p for _, p in manual)

print(f"{'layer':<20}{'params':>10}")
print("-" * 30)
for name, p in manual:
    print(f"{'name':<20}{'p':>10}")
print("-" * 30)
print(f"{'TOTAL (by hand)':<20}{'EXPECTED_PARAMS':>10}")
print(f"\nshared hyperparameters: epochs={EPOCHS}, batch_size={BATCH_SIZE},
    ↪ optimizer=Adam(lr={LR})")

results = [] # collects one U.RunResult per implementation
```

| layer           | params |
|-----------------|--------|
| -----           | -----  |
| Linear 179->128 | 23,040 |
| ReLU            | 0      |
| Dropout 0.3     | 0      |
| Linear 128->64  | 8,256  |
| ReLU            | 0      |
| Linear 64->3    | 195    |
| -----           | -----  |
| TOTAL (by hand) | 31,491 |

shared hyperparameters: epochs=15, batch\_size=256, optimizer=Adam(lr=0.001)

### 3. Cách cài đặt A — viết từ đầu (NumPy)

Không framework nào cả. `scratch_nn.py` định nghĩa mỗi tầng là một hàm tương minh kèm đạo hàm tự dẫn bằng tay, còn vòng lặp huấn luyện viết rõ cả bốn bước của chu trình:

Lan truyền xuôi → Hàm mất mát → Gradient → Cập nhật

Adam cũng được viết ra thành phép tính chứ không gọi từ thư viện, nên không có chi tiết nào của quy tắc cập nhật bị giấu đi.

```
U.set_seed(U.SEED)

scratch_model = S.Sequential([
```

```

    S.Dense(N_FEATURES, HIDDEN1, seed=1),
    S.ReLU(),
    S.Dropout(DROPOUT, seed=11),
    S.Dense(HIDDEN1, HIDDEN2, seed=2),
    S.ReLU(),
    S.Dense(HIDDEN2, N_CLASSES, seed=3),
])

print(scratch_model.summary(input_shape=(N_FEATURES,)))
print(f"\nmatches hand count {EXPECTED_PARAMS:,}? "
      f"{scratch_model.n_params() == EXPECTED_PARAMS}")

```

| layer             | output shape | params |
|-------------------|--------------|--------|
| Dense(179 -> 128) | (128,)       | 23,040 |
| ReLU              | (128,)       | 0      |
| Dropout(p=0.3)    | (128,)       | 0      |
| Dense(128 -> 64)  | (64,)        | 8,256  |
| ReLU              | (64,)        | 0      |
| Dense(64 -> 3)    | (3,)         | 195    |
| total             |              | 31,491 |

matches hand count 31,491? True

### Đạo hàm viết tay có đúng thật không?

Trước khi tin bất cứ con số nào mô hình này báo, gradient giải tích được đối chiếu với sai phân hữu hạn trung tâm,

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta_i + \epsilon) - \mathcal{L}(\theta_i - \epsilon)}{2\epsilon}$$

trên một lô nhỏ, tính ở độ chính xác float64. Sai số tương đối dưới khoảng 1e-4 nghĩa là lượt lan truyền ngược dẫn bằng tay khớp với đạo hàm số — hai nhánh dùng framework được autodiff cho sẵn tính đúng đắn này, còn nhánh scratch thì phải tự chứng minh.

```

U.set_seed(U.SEED)
check_model = S.Sequential([
    S.Dense(N_FEATURES, 16, seed=1),
    S.ReLU(),
    S.Dense(16, N_CLASSES, seed=3),
])
err = S.gradient_check(check_model, X_train[:8], y_train[:8], n_samples=8)
print(f"worst relative error vs finite differences: {err:.3e}")
print("PASS – hand-derived gradients agree with the numerical derivative" if err < 1e-4
      else "FAIL – backward pass disagrees with finite differences")

```

```

worst relative error vs finite differences: 4.511e-07
PASS – hand-derived gradients agree with the numerical derivative

```

```

U.set_seed(U.SEED)
with U.Timer() as t_scratch:
    hist_scratch = S.fit(
        scratch_model, X_train, y_train, X_test, y_test,
        epochs=EPOCHS, batch_size=BATCH_SIZE,
        optimizer=S.Adam(LR), seed=U.SEED,
    )
print(f"\ntotal training time: {t_scratch.seconds:.1f}s")

```

```

epoch 1/15  loss 0.9719  0.4s  test_acc 0.6003

```

```
epoch 2/15  loss 0.8816  0.4s  test_acc 0.6042

epoch 3/15  loss 0.8646  0.4s  test_acc 0.6045

epoch 4/15  loss 0.8555  0.4s  test_acc 0.6065

epoch 5/15  loss 0.8476  0.4s  test_acc 0.6089

epoch 6/15  loss 0.8467  0.4s  test_acc 0.6093

epoch 7/15  loss 0.8424  0.4s  test_acc 0.6093

epoch 8/15  loss 0.8384  0.4s  test_acc 0.6092

epoch 9/15  loss 0.8368  0.4s  test_acc 0.6086

epoch 10/15  loss 0.8346  0.4s  test_acc 0.6063

epoch 11/15  loss 0.8317  0.4s  test_acc 0.6071

epoch 12/15  loss 0.8292  0.3s  test_acc 0.6100

epoch 13/15  loss 0.8282  0.4s  test_acc 0.6086

epoch 14/15  loss 0.8253  0.4s  test_acc 0.6073

epoch 15/15  loss 0.8237  0.4s  test_acc 0.6094

total training time: 6.0s
```

```
pred = scratch_model.predict_classes(X_test)
m = U.evaluate(y_test, pred, N_CLASSES)

results.append(U.RunResult(
    framework="Scratch (NumPy)", dataset=meta["name"], model="MLP 179-128-64-3",
    n_params=scratch_model.n_params(), epochs=EPOCHS,
    train_seconds=round(t_scratch.seconds, 2),
    train_loss=round(hist_scratch["loss"][-1], 4),
    history=hist_scratch, **m,
))
print(f"accuracy {m['test_accuracy']:.4f}    macro-F1 {m['f1_macro']:.4f}")
```

```
accuracy 0.6094    macro-F1 0.3450
```

## 4. Cách cài đặt B — TensorFlow / Keras

Vẫn hợp thành hàm ấy, nhưng viết thành một danh sách các tầng dùng lại được. Hãy để ý những gì **biến mất** khỏi mã nguồn của ta: vòng lặp tường minh trên từng lô, phép tính mất mát, gradient và bước cập nhật tham số — tất cả nay nằm bên trong `fit()`.

***Lưu ý về phần cứng.** Bản TensorFlow chạy trực tiếp trên Windows không còn hỗ trợ GPU kể từ phiên bản 2.10, nên nhánh này chạy trên CPU trong khi nhánh PyTorch bên dưới dùng GPU. Vì vậy thời gian huấn luyện giữa hai dòng đó **không** so sánh trực tiếp được — số tham số và độ chính xác thì có.*

```
U.set_seed(U.SEED)

keras_model = keras.Sequential([
    keras.layers.Input(shape=(N_FEATURES,)),
    keras.layers.Dense(HIDDEN1, activation="relu"),
    keras.layers.Dropout(DROPOUT),
    keras.layers.Dense(HIDDEN2, activation="relu"),
    keras.layers.Dense(N_CLASSES),          # raw logits, like the other two legs
], name="diabetes_mlp")

keras_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=LR),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"],
)
keras_model.summary()
print(f"\nmatches hand count {EXPECTED_PARAMS:,}? "
      f"{keras_model.count_params() == EXPECTED_PARAMS}")
```

matches hand count 31,491? True

```
U.set_seed(U.SEED)
with U.Timer() as t_keras:
    hk = keras_model.fit(
        X_train, y_train,
        validation_data=(X_test, y_test),
        epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=2,
    )
print(f"\ntotal training time: {t_keras.seconds:.1f}s")
```

Epoch 1/15

219/219 - 1s - 6ms/step - accuracy: 0.5829 - loss: 0.9071 - val\_accuracy: 0.6032 - val\_loss: 0.8665

Epoch 2/15

219/219 - 0s - 2ms/step - accuracy: 0.5995 - loss: 0.8653 - val\_accuracy: 0.6078 - val\_loss: 0.8586

Epoch 3/15

219/219 - 1s - 2ms/step - accuracy: 0.6072 - loss: 0.8531 - val\_accuracy: 0.6066 - val\_loss: 0.8552

Epoch 4/15

219/219 - 1s - 2ms/step - accuracy: 0.6097 - loss: 0.8479 - val\_accuracy: 0.6085 - val\_loss: 0.8529

Epoch 5/15

219/219 - 0s - 2ms/step - accuracy: 0.6113 - loss: 0.8415 - val\_accuracy: 0.6060 - val\_loss: 0.8523

Epoch 6/15

219/219 - 0s - 2ms/step - accuracy: 0.6138 - loss: 0.8383 - val\_accuracy: 0.6095 - val\_loss: 0.8519

Epoch 7/15

219/219 - 0s - 2ms/step - accuracy: 0.6146 - loss: 0.8362 - val\_accuracy: 0.6096 - val\_loss: 0.8516

Epoch 8/15

219/219 - 0s - 2ms/step - accuracy: 0.6171 - loss: 0.8330 - val\_accuracy: 0.6090 - val\_loss: 0.8521

Epoch 9/15

219/219 - 0s - 2ms/step - accuracy: 0.6161 - loss: 0.8296 - val\_accuracy: 0.6090 - val\_loss: 0.8526

Epoch 10/15

219/219 - 0s - 2ms/step - accuracy: 0.6187 - loss: 0.8291 - val\_accuracy: 0.6098 - val\_loss: 0.8520

Epoch 11/15

219/219 - 0s - 2ms/step - accuracy: 0.6206 - loss: 0.8252 - val\_accuracy: 0.6104 - val\_loss: 0.8529

Epoch 12/15

219/219 - 0s - 2ms/step - accuracy: 0.6211 - loss: 0.8236 - val\_accuracy: 0.6116 - val\_loss: 0.8519

Epoch 13/15

219/219 - 0s - 2ms/step - accuracy: 0.6218 - loss: 0.8215 - val\_accuracy: 0.6065 - val\_loss: 0.8528

Epoch 14/15

219/219 - 0s - 2ms/step - accuracy: 0.6222 - loss: 0.8193 - val\_accuracy: 0.6074 - val\_loss: 0.8542

Epoch 15/15

219/219 - 0s - 2ms/step - accuracy: 0.6249 - loss: 0.8174 - val\_accuracy: 0.6077 - val\_loss: 0.8549

total training time: 7.8s

```
pred = keras_model.predict(X_test, batch_size=512, verbose=0).argmax(axis=1)
m = U.evaluate(y_test, pred, N_CLASSES)

hist_keras = {"loss": [float(v) for v in hk.history["loss"]],
              "val_acc": [float(v) for v in hk.history["val_accuracy"]]}

results.append(U.RunResult(
    framework="TensorFlow/Keras", dataset=meta["name"], model="MLP 179-128-64-3",
    n_params=int(keras_model.count_params()), epochs=EPOCHS,
    train_seconds=round(t_keras.seconds, 2),
    train_loss=round(hist_keras["loss"][-1], 4),
    history=hist_keras, **m,
))
print(f"accuracy {m['test_accuracy']:.4f}    macro-F1 {m['f1_macro']:.4f}")
```

accuracy 0.6077 macro-F1 0.3686

## 5. Cách cài đặt C — PyTorch

Lại vẫn hợp thành hàm ấy, lần này dưới dạng một `nn.Module`. PyTorch nằm giữa hai cách kia: phần **mô hình** gọn như Keras, còn phần **vòng lặp huấn luyện** tường minh như bản scratch —

Xoá gradient → Lan truyền xuôi → Mất mát → Lan truyền ngược → Cập nhật

được viết ra từng dòng. Thử PyTorch cho sẵn mà bản scratch phải tự dẫn bằng tay chính là `loss.backward(): autograd`.

```
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

U.set_seed(U.SEED)

class DiabetesMLP(nn.Module):
    def __init__(self, n_in, n_classes):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_in, HIDDEN1),
            nn.ReLU(),
            nn.Dropout(DROPOUT),
            nn.Linear(HIDDEN1, HIDDEN2),
            nn.ReLU(),
            nn.Linear(HIDDEN2, n_classes),
        )

    def forward(self, x):
        return self.net(x)

torch_model = DiabetesMLP(N_FEATURES, N_CLASSES).to(DEVICE)
```

```
n_torch = sum(p.numel() for p in torch_model.parameters())

print(torch_model)
print(f"\nparameters: {n_torch:,}")
print(f"matches hand count {EXPECTED_PARAMS:,}? {n_torch == EXPECTED_PARAMS}")
```

```
DiabetesMLP(
  (net): Sequential(
    (0): Linear(in_features=179, out_features=128, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.3, inplace=False)
    (3): Linear(in_features=128, out_features=64, bias=True)
    (4): ReLU()
    (5): Linear(in_features=64, out_features=3, bias=True)
  )
)

parameters: 31,491
matches hand count 31,491? True
```

```
train_ds = TensorDataset(torch.tensor(X_train), torch.tensor(y_train))
train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True)

Xte_t = torch.tensor(X_test).to(DEVICE)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(torch_model.parameters(), lr=LR)

hist_torch = {"loss": [], "val_acc": []}

with U.Timer() as t_torch:
    for epoch in range(1, EPOCHS + 1):
        torch_model.train()
        running, nb = 0.0, 0
        for xb, yb in train_loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)

            optimizer.zero_grad()          # 1. zero gradient
            logits = torch_model(xb)        # 2. forward
            loss = criterion(logits, yb)    # 3. loss
            loss.backward()                 # 4. backward (autograd)
            optimizer.step()                # 5. update

            running += loss.item(); nb += 1

        torch_model.eval()
        with torch.no_grad():
            acc = (torch_model(Xte_t).argmax(1).cpu().numpy() == y_test).mean()

        hist_torch["loss"].append(running / nb)
        hist_torch["val_acc"].append(float(acc))
        print(f"epoch {epoch}/{EPOCHS}  loss {running/nb:.4f}  test_acc {acc:.4f}")

print(f"\ntotal training time: {t_torch.seconds:.1f}s")
```

```
epoch 1/15  loss 0.8733  test_acc 0.6093
```

```
epoch 2/15  loss 0.8493  test_acc 0.6103
```

```
epoch 3/15  loss 0.8421  test_acc 0.6095
```

```
epoch 4/15  loss 0.8402  test_acc 0.6083

epoch 5/15  loss 0.8363  test_acc 0.6105

epoch 6/15  loss 0.8334  test_acc 0.6109

epoch 7/15  loss 0.8306  test_acc 0.6084

epoch 8/15  loss 0.8286  test_acc 0.6107

epoch 9/15  loss 0.8255  test_acc 0.6043

epoch 10/15  loss 0.8240  test_acc 0.6086

epoch 11/15  loss 0.8201  test_acc 0.6110

epoch 12/15  loss 0.8191  test_acc 0.6108

epoch 13/15  loss 0.8161  test_acc 0.6095

epoch 14/15  loss 0.8155  test_acc 0.6121

epoch 15/15  loss 0.8135  test_acc 0.6105

total training time: 9.2s
```

```
torch_model.eval()
with torch.no_grad():
    pred = torch_model(Xte_t).argmax(1).cpu().numpy()
m = U.evaluate(y_test, pred, N_CLASSES)

results.append(U.RunResult(
    framework="PyTorch", dataset=meta["name"], model="MLP 179-128-64-3",
    n_params=n_torch, epochs=EPOCHS,
    train_seconds=round(t_torch.seconds, 2),
    train_loss=round(hist_torch["loss"][-1], 4),
    history=hist_torch, **m,
))
print(f"accuracy {m['test_accuracy']:.4f}    macro-F1 {m['f1_macro']:.4f}")
```

```
accuracy 0.6105    macro-F1 0.3694
```

## 6. So sánh

### 6.1 Những con số slide 29 yêu cầu

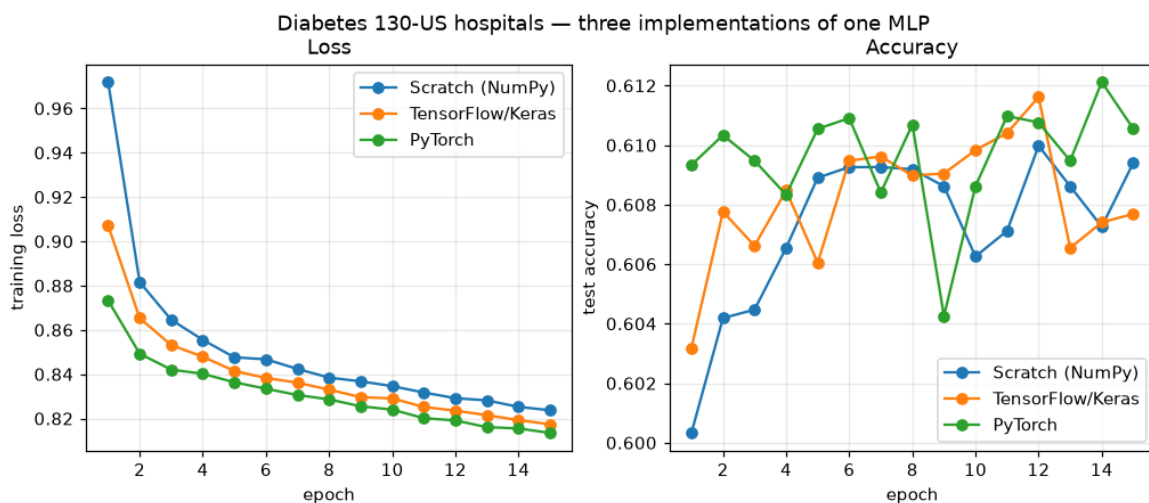
```
table = U.results_table(results)
display(table)
```

|   | framework        | dataset                   | model            | n_params | epochs | train_seconds | train_loss | test_accuracy | precision_macro | recall_macro | f1_macro |
|---|------------------|---------------------------|------------------|----------|--------|---------------|------------|---------------|-----------------|--------------|----------|
| 0 | Scratch (NumPy)  | Diabetes 130-US hospitals | MLP 179-128-64-3 | 31491    | 15     | 6.04          | 0.8237     | 0.6094        | 0.4798          | 0.3728       | 0.3450   |
| 1 | TensorFlow/Keras | Diabetes 130-US hospitals | MLP 179-128-64-3 | 31491    | 15     | 7.78          | 0.8174     | 0.6077        | 0.4839          | 0.3842       | 0.3686   |
| 2 | PyTorch          | Diabetes 130-US hospitals | MLP 179-128-64-3 | 31491    | 15     | 9.17          | 0.8135     | 0.6105        | 0.5129          | 0.3860       | 0.3694   |

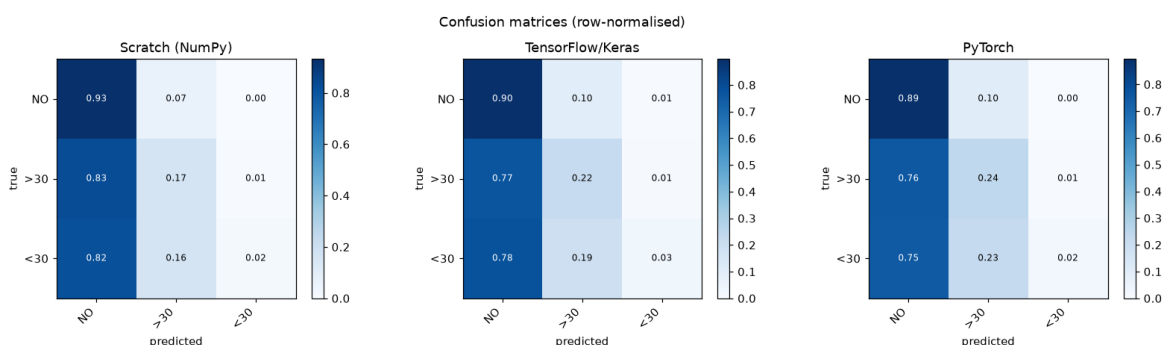
```
# A majority-class baseline, so the accuracies above can be read honestly.
majority = np.bincount(y_train).argmax()
base = U.evaluate(y_test, np.full_like(y_test, majority), N_CLASSES)
print(f"always-predict-'{CLASSES[majority]}' baseline: "
      f"accuracy {base['test_accuracy']:.4f}    macro-F1 {base['f1_macro']:.4f}")
print(f"best model: "
      f"accuracy {table['test_accuracy'].max():.4f}    macro-F1 "
      f"↪ {table['f1_macro'].max():.4f}")
```

```
always-predict-'NO' baseline: accuracy 0.5958    macro-F1 0.2489
best model:                accuracy 0.6105    macro-F1 0.3694
```

```
U.plot_history({r.framework: r.history for r in results},
              title=f"{meta['name']} - three implementations of one MLP")
plt.show()
```



```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.2))
for ax, r in zip(axes, results):
    U.plot_confusion(r.confusion, CLASSES, title=r.framework, ax=ax)
fig.suptitle("Confusion matrices (row-normalised)")
fig.tight_layout()
plt.show()
```



## 6.2 Mỗi thành phần nằm ở đâu

Đây là bảng mà slide 28 yêu cầu sinh viên nộp, điền theo đúng notebook này.

```

component_table = pd.DataFrame([
    ["Data loading", "ass4_utils -> NumPy arrays", "same arrays", "same
↪ arrays"],
    ["Preprocessing", "ass4_utils (shared)", "shared", "shared"],
    ["Model definition", "S.Sequential([...])", "keras.Sequential([...])",
↪ "nn.Module + nn.Sequential"],
    ["Dense layer", "S.Dense (x@W+b by hand)", "keras.layers.Dense",
↪ "nn.Linear"],
    ["Activation", "S.ReLU (mask multiply)", "activation='relu'",
↪ "nn.ReLU"],
    ["Regularisation", "S.Dropout (inverted)", "keras.layers.Dropout",
↪ "nn.Dropout"],
    ["Loss", "S.SoftmaxCrossEntropy", "SparseCategoricalCrossentropy",
↪ "nn.CrossEntropyLoss"],
    ["Gradient", "hand-derived backward()", "automatic (GradientTape)",
↪ "automatic (autograd)"],
    ["Optimizer", "S.Adam (written out)", "keras.optimizers.Adam",
↪ "torch.optim.Adam"],
    ["Training loop", "explicit for-loop", "model.fit()", "explicit
↪ for-loop"],
    ["Evaluation", "predict_classes + sklearn", "predict + sklearn", "forward
↪ + sklearn"],
], columns=["Component", "Scratch", "TensorFlow/Keras", "PyTorch"])

display(component_table)

```

|    | Component        | Scratch                    | TensorFlow/Keras              | PyTorch                      |
|----|------------------|----------------------------|-------------------------------|------------------------------|
| 0  | Data loading     | ass4_utils -> NumPy arrays | same arrays                   | same arrays                  |
| 1  | Preprocessing    | ass4_utils (shared)        | shared                        | shared                       |
| 2  | Model definition | S.Sequential([...])        | keras.Sequential([...])       | nn.Module +<br>nn.Sequential |
| 3  | Dense layer      | S.Dense (x@W+b by hand)    | keras.layers.Dense            | nn.Linear                    |
| 4  | Activation       | S.ReLU (mask multiply)     | activation='relu'             | nn.ReLU                      |
| 5  | Regularisation   | S.Dropout (inverted)       | keras.layers.Dropout          | nn.Dropout                   |
| 6  | Loss             | S.SoftmaxCrossEntropy      | SparseCategoricalCrossentropy | nn.CrossEntropyLoss          |
| 7  | Gradient         | hand-derived backward()    | automatic (GradientTape)      | automatic (autograd)         |
| 8  | Optimizer        | S.Adam (written out)       | keras.optimizers.Adam         | torch.optim.Adam             |
| 9  | Training loop    | explicit for-loop          | model.fit()                   | explicit for-loop            |
| 10 | Evaluation       | predict_classes + sklearn  | predict + sklearn             | forward + sklearn            |

```

# Persist for 04_compare.ipynb
os.makedirs("results", exist_ok=True)
with open("results/01_diabetes130.json", "w") as f:
    json.dump({"meta": {k: v for k, v in meta.items() if k != "feature_names"},
               "results": [r.__dict__ for r in results],
               "component_table": component_table.to_dict(orient="records")},
              f, indent=2, default=str)
print("saved results/01_diabetes130.json")

```

saved results/01\_diabetes130.json

## Lưu lại mô hình đã huấn luyện

Tập JSON ở trên ghi lại các mô hình **đạt** bao nhiêu điểm. Nó không ghi lại các mô hình **là** gì — muốn có lại những con số này thì phải huấn luyện lại từ đầu.

`U.save_model` ghi từng nhánh theo định dạng gốc của framework tương ứng, vào `results/models/<notebook>/`, kèm một tập JSON đi cùng ghi rõ kiến trúc và điểm số mà đúng bộ trọng số đó đạt được:

| nhánh            | tệp                                        | tự nạp lại được? |
|------------------|--------------------------------------------|------------------|
| Scratch (NumPy)  | .npz, mỗi tham số của mỗi tầng là một mảng | không            |
| TensorFlow/Keras | .keras, đồ thị + trọng số                  | <b>được</b>      |
| PyTorch          | .pt, state_dict                            | không            |

.pt và .npz chỉ chứa trọng số, nên nạp lại chúng đồng nghĩa với việc phải dựng lại kiến trúc trước rồi truyền đối tượng mới vào `U.load_model(..., model=...)`. Đây đúng là giao kèo mà `variants.py` dùng cho `results/variant_cache/`.

```
# Weights are the model; metrics only describe it. Persist all three legs.
TAG = "01_diabetes130"

by_fw = {r.framework: r for r in results}

for obj, fw, stem in [
    (scratch_model, "Scratch (NumPy)", "mlp_scratch"),
    (keras_model, "TensorFlow/Keras", "mlp_keras"),
    (torch_model, "PyTorch", "mlp_torch"),
]:
    r = by_fw[fw]
    U.save_model(obj, f"{stem}", TAG, architecture=r.model, dataset=r.dataset,
                epochs=r.epochs, test_accuracy=r.test_accuracy, f1_macro=r.f1_macro)

display(U.list_models(TAG)[["name", "framework", "n_params", "dataset", "test_accuracy"]])

saved results\models\01_diabetes130\mlp_scratch.npz (31,491 params, 116 KB)
saved results\models\01_diabetes130\mlp_keras.keras (31,491 params, 396 KB)
saved results\models\01_diabetes130\mlp_torch.pt (31,491 params, 126 KB)
```

|   | name        | framework | n_params | dataset                   | test_accuracy |
|---|-------------|-----------|----------|---------------------------|---------------|
| 0 | mlp_keras   | keras     | 31491    | Diabetes 130-US hospitals | 0.607689      |
| 1 | mlp_scratch | scratch   | 31491    | Diabetes 130-US hospitals | 0.609404      |
| 2 | mlp_torch   | torch     | 31491    | Diabetes 130-US hospitals | 0.610547      |

## 7. Bộ dữ liệu này cho thấy điều gì

**Ba cách cài đặt cho kết quả thống nhất.** Cả ba đều dựng cùng một hàm 31.491 tham số, và cả ba đều rơi vào trong dải nhiễu của nhau ở độ chính xác lần macro F1. Tên gọi khác nhau — `S.Dense`, `keras.layers.Dense`, `nn.Linear` — không kéo theo khái niệm học máy khác nhau. Mức trừu tượng đối; phần toán thì không.

**Ở đây độ chính xác không phải con số đáng đọc nhất.** Với tỷ lệ lớp 59 / 32 / 9, mức sàn “luôn đoán NO” đã sát với mức các mạng đã huấn luyện đạt được. Cột macro F1 mới là thứ phân biệt mô hình học được điều gì đó với mô hình chỉ học thuộc phân phối tiên nghiệm. Dự đoán tái nhập viện từ dữ liệu hành chính vốn khó thật — các công bố trên bộ dữ liệu này báo AUC quanh 0,65, và không có gì ở đây mâu thuẫn với điều đó.

**Ca dữ liệu bằng là ca đối chứng.** Không có phép tích chập nào xuất hiện trong notebook này, vì không có lân cận không gian nào để một hạt nhân (kernel) khai thác. Chính điều đó làm hai notebook sau có ý nghĩa: khi MNIST và CIFAR-10 đưa Conv2D vào, đó là phản ứng trước một tính chất dữ liệu thực sự có, không phải lựa chọn mặc định.

**GPU không giúp được gì ở đây — và chỗ này đáng dừng lại một nhịp.** Trên bộ dữ liệu này, nhánh PyTorch là nhánh **chậm nhất** dù chạy trên RTX 3060, còn nhánh NumPy viết tay lại nhanh nhất. Mô hình thì bé, dữ liệu dạng bảng, nên mỗi lô mang quá ít phép tính để bù lại chi phí chuyển dữ liệu sang thiết bị rồi chuyển về; phần chi phí truyền dữ liệu lấn át tất cả. Tăng tốc phần cứng không miễn phí, và cũng không tự động. Hai notebook ảnh hưởng đúng điều kiện mà nó cần — đủ khối lượng tính toán trên mỗi lô để bù chi phí — và ở đó thứ tự đảo ngược rõ rệt.

**Nhánh scratch đổi lại được gì.** Nó là nhánh duy nhất phải chứng minh gradient của mình đúng thay vì thừa hưởng từ autodiff. Đổi lại, mọi đại lượng trong chu trình huấn luyện đều nhìn thấy được — và phép kiểm tra sai phân hữu hạn ở trên là bằng chứng rằng “nhìn thấy được” cũng có nghĩa là “đúng”.

Tiếp theo: `02_mnist.ipynb`, nơi dữ liệu có cấu trúc không gian và kiến trúc đổi theo cho khớp.